
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON VAULT - Requirements & Grading Criteria

ARM Cortex-M33 • GDB • Ghidra

September 21, 2026

For educational and defensive security research only

OPERATION IRON VAULT

REQUIREMENTS & GRADING CRITERIA

TARGET: NorthPharma datacenter vent controller (server hall cooling node)
ARTIFACT: ACT-VIII.bin / ACT-VIII.uf2 (compromised)
CREW: FROSTLINE OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma runs the buildings that keep the state's data alive, and its datacenter vent controller built on a Pico 2 is the node on the edge of the cooling plant. A contractor called **FROSTLINE** planted a locker in the node image: an inverted vent lock that forces the damper closed, an inverted display mask that renders `ST:MAINT`, a reserved-sector lock marker that re-arms the lock on every boot, and an inverted vent command authorization verdict. Operative **NIGHTINGALE** recovered the compromised image as `ACT-VIII.bin`.

Students are the reverse-engineering reserve. They reverse engineer `ACT-VIII.bin` with Ghidra, find and patch all four defects, defeat the CoreDebug `DHCSR` anti-debug under GDB to observe the marker write, export a corrected image, flash it to a real Pico 2, and prove the corrected behavior on the breadboard. The machine check is `scripts/verify_ctf.py`.

The challenge is a standalone capstone exercise and contains no answer, constant, address, bug, or patch belonging to any other course assignment.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the monitor loop.
- Locate four corrupted bytes: a forced-close lock gate, a display mask gate, a reserved-sector marker gate, and an authorization verdict branch.
- Analyze `cbz` and `cbnz` condition semantics and branch inversion.
- Explain why a local lock that overrides the output makes an authenticated command path irrelevant, and why an availability attack does not need the cipher.
- Explain why a device that misreports its own state hides a lockout behind the appearance of routine maintenance.
- Explain why reserved-flash state survives a firmware reflash.
- Read CoreDebug `DHCSR`, explain the anti-debug trap, and defeat it under GDB.
- Explain why authentication is not authorization and why a verdict must be verified before the command is applied.

Students must use only the course concepts: ARM registers, stack behavior, USB-CDC and UART consoles, GDB, Ghidra static analysis and binary patching, vector tables, reset startup, XIP, Thumb addressing, condition-code analysis, stateful security, and the Argon2id plus XChaCha20-Poly1305 authenticated envelope.

Deliverables Checklist

#	Deliverable	Format	Criterion
1	Ghidra project screenshot	PNG/JPG	Task 1
2	Vector table and boot table	Inside <code>ACT-VIII-Answers.md</code>	Task 1
3	<code>main</code> and monitor-loop table	Inside <code>ACT-VIII-Answers.md</code>	Task 1
4	Module map	Inside <code>ACT-VIII-Answers.md</code>	Task 1
5	Vent lock evidence and patch	Inside <code>ACT-VIII-Answers.md</code>	Task 2
6	LCD mask evidence and patch	Inside <code>ACT-VIII-Answers.md</code>	Task 3
7	Anti-debug GDB proof, reserved-sector evidence, and patch	Inside <code>ACT-VIII-Answers.md</code>	Task 4
8	Vent command authorization evidence and patch	Inside <code>ACT-VIII-Answers.md</code>	Task 5
9	<code>ACT-VIII_fixed.bin</code>	BIN file	Task 6
10	<code>ACT-VIII_fixed.uf2</code>	UF2 file	Task 6
11	Hardware proof and reflection	Inside <code>ACT-VIII-Answers.md</code>	Task 6

Required Tools and Equipment

Tool	Purpose
Raspberry Pi Pico 2	Isolated target node
Debug Probe (OpenOCD)	SWD connection for GDB inspection and the anti-debug work
arm-none-eabi-gdb	Runtime breakpoints, <code>DHCSR</code> clearing, and reserved-sector observation
Ghidra	Static analysis and binary patching
Python 3 with <code>uf2conv.py</code>	UF2 conversion and artifact checks
DHT11, 1602 I2C LCD, RYL998, IR receiver, SG90 servo, 3 LEDs, manual purge button	Breadboard hardware proof
<code>ACT-VIII.bin</code> and <code>ACT-VIII.uf2</code>	Supplied compromised artifacts

Console settings: **USB-CDC virtual COM port, 115200 baud, 8 data bits, no parity, 1 stop bit.** Radio UART settings: **UART1, 115200, network ID 18.**

Artifact Identity

The instructor-issued artifact hashes are:

```

ACT-VIII.bin      3eb95ef74dcc11bc5a9b77b48645390f17d969c1832a3a7f765ebad7ebd0e120
ACT-VIII.uf2      b653b2b3c1542caf82a7c03d26dd290dc8859882780c2dfdf799d2d69872238d
ACT-VIII_fixed.bin 59411f4bf07809ada695315d75f43c47b7a6efa7c055c8c46841cc6ae0caa234
ACT-VIII_fixed.uf2 4443ce43225991d358e20ab409310209427da83977f6bdf59135a13701ba4a2b

```

The verifier checks the `ACT-VIII.bin` and `ACT-VIII_fixed.bin` hashes specifically, asserts the four fixed bytes, and requires that only those four offsets differ between the two `.bin` images. Both `.bin` images are 50,860 bytes and both `.uf2` images are 102,400 bytes.

Grading Rubric - Detailed Breakdown

Task 1: Setup and Initial Analysis (10 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project <code>IronVault_Investigation</code> , raw binary import	One item off	Not set up
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor	Wrong language	Missing
[DOCUMENT] Base address set to 0x10000000	2	Base <code>0x10000000</code>	Wrong base	Missing
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base <code>0x10000000</code> , initial SP <code>0x20082000</code> , reset handler <code>0x1000015D</code>	One missing	Not found
[DOCUMENT] main and the vent monitor state machine (monitor_step) addresses identified	1	main <code>0x10000234</code> , monitor_step <code>0x10006648</code>	One correct	Neither
[DOCUMENT] Module map identifies the damper, control, vault_auth, implant, and monitor anchors	1	At least one correct anchor per module	Partial	Missing

Task 2: Bug #1 The Vent Lock (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the vent lock gate at 0x1000A33B	5	Address and function (<code>implant_init</code> , inlined lock gate) identified	Approximate	Not found
[DOCUMENT] Documented the forced close that holds the vent shut and lights LOCKED	5	Lock gate <code>0x20013CF2</code> , forced closed damper, yellow LOCKED lamp	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xBB to 0xB3 so the vent lock is not armed	7	Byte <code>0xBB</code> changed to <code>0xB3</code>	Wrong byte	Not patched
[DOCUMENT] Explained why the lock forces the vent closed regardless of the authorized state	3	Local override beside the authenticated command path, availability as a policy control	Vague	Missing

Task 3: Bug #2 The LCD Mask (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the mask gate at 0x1000A311	5	Address and function (<code>implant_mask_active</code>) identified	Approximate	Not found
[DOCUMENT] Documented the ST:MAINT maintenance mask that hides the true vent state	5	Mask gate <code>0x20013CF5</code> , <code>monitor_state_text</code> returns <code>MAINT</code> while locked	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so the true state is never masked	7	Byte <code>0xB9</code> changed to <code>0xB1</code>	Wrong byte	Not patched
[DOCUMENT] Explained why a masked state hides an availability failure	3	The readout is part of the attack surface and the truth is a control	Vague	Missing

Task 4: Bug #3 The Lock Marker (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the lock marker gate at 0x1000A353	5	Address and inlined <code>implant_init</code> path identified	Approximate	Not found
[DOCUMENT] Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB	5	<code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so no lock marker is programmed to 0x103FF000	7	Byte <code>0xB9</code> changed to <code>0xB1</code>	Wrong byte	Not patched
[DOCUMENT] Explained the reserved sector 0x103FF000 and the lock marker byte 0x4C	3	Marker, reserved sector, write-once first run	Vague	Missing

Task 5: Bug #4 The Vent Command Authorization (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the vent command authorization branch at 0x10007595	5	Address and function (<code>control_handle_frame</code>) identified	Approximate	Not found
[DOCUMENT] Documented the authorization verdict inversion and the branch condition	5	Reject when the verdict is false	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so failed and replayed authorizations are rejected	7	Byte <code>0xB9</code> changed to <code>0xB1</code>	Wrong byte	Not patched

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Explained why an unauthenticated or replayed vent command must be rejected	3	The applied command must see only an authorized verdict	Vague	Missing

Task 6: Export and Verify (10 points)

Criterion	Points	Full credit	Partial credit	No credit
[PATCH] Exported ACT-VIII_fixed.bin from Ghidra	2	Valid patched binary	Corrupt	Not submitted
[PATCH] Converted to ACT-VIII_fixed.uf2 with the correct base and family	2	<code>--base 0x10000000 --family 0xe48bff59</code>	Wrong flags	Not submitted
[DOCUMENT] scripts/verify_ctf.py passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown	Partial proof	No proof
[DOCUMENT] Reflection maps each of the four defects to a real-world control-system failure	3	Specific mapping for all four	Partial	Missing

Common Pitfalls

Pitfall	Consequence	Avoidance
Reading the vent lock gate backwards	The vent is still forced closed and the LOCKED lamp is lit	Neutralize only on the clear-gate branch (<code>cbz</code> , <code>0xB3</code>)
Reading the mask gate backwards	The readout still shows <code>ST:MAINT</code>	Neutralize only when the gate is clear (<code>cbz</code> , <code>0xB1</code>)
Confusing <code>cbz</code> and <code>cbnz</code> at <code>0xA311</code> or <code>0xA353</code>	The mask still lies, or the marker is still written	Neutralize only when the gate is clear (<code>cbz</code> , <code>0xB1</code>)
Patching the low byte at <code>0xA33A</code> , <code>0xA310</code> , <code>0xA352</code> , or <code>0x7594</code>	The condition code never changes	Patch the high byte at <code>0xA33B</code> , <code>0xA311</code> , <code>0xA353</code> , <code>0x7595</code>
Searching for a standalone <code>implant_infect</code> symbol	Cannot find the inlined gate	Look inside <code>implant_init</code> at <code>0xA353</code>
Confusing the lock gate with the marker gate	Both sit in <code>implant_init</code> at <code>0xA33B</code> and <code>0xA353</code>	Patch the lock gate first, then the marker gate
Patching the shipped image before observing the write	You never prove the lock marker write	Defeat <code>DHCSR</code> under GDB first, then patch the artifact
Fabricating the GDB session	Verification fails	Show the command sequence and the real observed code path
Treating the anti-debug as a defect to patch	Wasted effort; it is identical in both images	Defeat it in a scratch copy or with GDB, then patch the real defect

Pitfall	Consequence	Avoidance
Missing that the authorization branch is a verdict	Unauthenticated commands still reach the applied command and zone	Accept only when the verdict is true (<code>cbz</code> to reject, <code>0xB1</code>)
Forgetting that the fix is also a policy	The device still fails closed on a lost link	Make the vent fail open when the link is lost
Forgetting UF2 conversion	Raw binary will not flash	Use <code>uf2conv.py</code> with family <code>0xe48bff59</code>

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 rack temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Vent damper servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	HALL HOT, 220 to 330 ohm to GND
Yellow LED	anode	GP17	LOCKED, 220 to 330 ohm to GND
Green LED	anode	GP18	COOLING OK, 220 to 330 ohm to GND
Manual purge button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply and the servo rail. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Memory Map Reference

Region	Address	Purpose
Bootrom	0x00000000	Immutable boot code
Flash/XIP	0x10000000	Vector table, code, rodata, data image
SRAM	0x20000000	Stack and writable state
CoreDebug_DHCSR	0xE00EDF0	Anti-debug register read by the locker
Locker reserved sector	0x103FF000	Lock marker target (last flash sector)
Locker tick counter	0x200136F4	Incremented once per <code>implant_tick</code>
Locker lock count	0x200136F0	Number of lock operations this boot
Locker active flag	0x20013CF1	Set when the locker arms
Locker locked flag	0x20013CF3	True while the vent is held closed
Locker lock gate	0x20013CF2	Gates the ransom vent lock
Locker mask gate	0x20013CF5	Gates the <code>ST:MAINT</code> display mask
Locker marker gate	0x20013CF4	Gates the reserved-sector lock marker write
Control ready gate	0x20013CED	Gates the sealed vent command path
Applied command	0x20013CEC	Command after a true verdict
Applied zone	0x20013CE2	Zone after a true verdict
Authorization ready gate	0x20013CFF	Gates the authorization check
Auth state record	0x200136AC	Anti-replay and state-tag record
Control field key	0x20013848	Derived field key for the envelope
Envelope workspace	0x200136C8	Sealed frame open workspace

The VA of any file offset is the file offset plus `0x10000000`.

Deadline & Submission

- Create a folder containing the Ghidra screenshot, `ACT-VIII_fixed.bin`, and `ACT-VIII_fixed.uf2`.
- Write all written answers in `ACT-VIII-Answers.md` inside that folder.
- Include the output of `python scripts/verify_ctf.py`.
- ZIP the folder as `lastname-firstname-ACT-VIII.zip`.
- Submit the ZIP before the posted deadline; late submissions lose 10 percent per day.

Grade Scale

Grade	Percentage	Points
A+	97-100%	97-100
A	93-96%	93-96
A-	90-92%	90-92
B+	87-89%	87-89
B	84-86%	84-86
B-	80-83%	80-83
C	70-79%	70-79
F	0-69%	0-69%

Academic Integrity

Use only the supplied Pico 2 and firmware. Do not connect the exercise to an operational datacenter network, a building-management system, a cooling plant control system, a public network, a military system, or a third-party device. This is a controlled, isolated educational exercise. All analysis and patches must be your own work; sharing binaries, addresses, keys, passphrases, or answers is a violation of the academic integrity policy.

Reference Material

Topic	Reference
ARM Cortex-M33 registers and stack	Course block 1
USB-CDC and UART console capture	Course block 2
Vector tables, reset startup, and XIP	Course block 3
Ghidra static analysis and binary patching	Course block 4
Availability attacks, lockout logic, and ransom logic	Course block 5
Display integrity and misleading annunciation	Course block 6
Reserved-flash persistence and boot re-install	Course block 7
CoreDebug <code>DHCSR</code> and anti-debug	Course block 8
Argon2id and XChaCha20-Poly1305 authenticated envelope	Course block 9